

Proximal Policy Optimization (PPO)

Pietro Cestola

1 Introduzione

Proximal Policy Optimization (PPO) è un algoritmo di *reinforcement learning* appartenente alla famiglia dei policy gradient methods. Introdotto da Schulman et al. (2017), PPO combina semplicità di implementazione, stabilità numerica e buone prestazioni. Rispetto ad approcci come Trust Region Policy Optimization (TRPO), PPO evita vincoli di ottimizzazione complessi e introduce un obiettivo *clipped* che limita variazioni troppo grandi nella policy tra aggiornamenti successivi. Per queste ragioni è oggi uno standard de facto in moltissimi campi.

2 Richiami di Reinforcement Learning

Definizione 1 (Processo decisionale di Markov). Un processo decisionale di Markov (MDP) è una tupla

$$M = (S, A, P, r, \gamma),$$

dove:

- S è l'insieme di tutti gli **stati** dell'ambiente;
- A è l'insieme di tutte le **azioni** possibili;
- $P(s'|s, a)$ è la funzione di **transizione**, ossia la probabilità che, trovandosi nello stato s ed eseguendo l'azione a , l'ambiente transiti nello stato s' ;
- $r(s, a)$ è la funzione di **ricompensa attesa**, cioè il valore medio della ricompensa immediata ottenuta eseguendo a in s ;
- $\gamma \in [0, 1]$ è il **fattore di sconto**, che determina l'importanza relativa delle ricompense future.

Definizione 2 (Policy). Una **policy** è una distribuzione di probabilità sulle azioni condizionata dallo stato, formalmente una mappa

$$\pi : S \times A \rightarrow [0, 1], \quad (s, a) \mapsto \pi(a|s),$$

tale che, per ogni stato $s \in S$, la funzione $a \mapsto \pi(a|s)$ sia una distribuzione di probabilità sulle azioni possibili A , ossia,

$$\sum_{a \in A} \pi(a|s) = 1.$$

Definizione 3 (Dinamica). Dato un MDP $M = (S, A, P, r, \gamma)$, una policy π ed uno stato iniziale s_0 estratto tramite una distribuzione ρ su S , si definisce **dinamica** la legge probabilistica che descrive l'evoluzione agente-ambiente:

- al passo t , l'agente osserva lo stato $s_t \in S$;
- seleziona un'azione $a_t \in A$ campionando da $\pi(\cdot|s_t)$;
- riceve la ricompensa $r_{t+1} = r(s_t, a_t)$;
- l'ambiente transita in un nuovo stato s_{t+1} secondo $P(\cdot|s_t, a_t)$.

Una **traiettoria** τ è una singola realizzazione della dinamica:

$$\tau = (s_0, a_0, r_1, s_1, a_1, r_2, \dots),$$

la probabilità di una traiettoria finita di lunghezza T è:

$$\mathbb{P}(\tau) = \rho(s_0) \prod_{t=0}^{T-1} \pi(a_t|s_t) P(s_{t+1}|s_t, a_t).$$

Definizione 4 (Ricompensa totale scontata). Dato un MDP $M = (S, A, P, r, \gamma)$ e una traiettoria $\tau = (s_0, a_0, r_1, s_1, a_1, r_2, \dots)$ generata secondo la dinamica associata ad una policy π , la **ricompensa totale scontata** a partire dal passo t è definita come

$$R_t = \sum_{k=t}^{\infty} \gamma^{k-t} r_{k+1},$$

ossia la somma, scontata nel tempo, di tutte le ricompense future ricevute dopo il tempo t . Per episodi finiti di lunghezza T , la somma è intesa fino al termine dell'episodio:

$$R_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_{k+1}.$$

Definizione 5 (Funzioni valore e vantaggio). Dato un MDP $M = (S, A, P, r, \gamma)$ ed una policy π si definiscono le funzioni di valore:

$$V^\pi(s) = \mathbb{E}_\pi[R_t | s_t = s], \quad Q^\pi(s, a) = \mathbb{E}_\pi[R_t | s_t = s, a_t = a],$$

e il **vantaggio**

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s).$$

3 Richiami di Policy Gradient Methods

I *policy gradient methods* sono una famiglia di algoritmi di reinforcement learning che ottimizzano direttamente la policy dell'agente per massimizzare il rendimento atteso. L'idea di base è trattare la policy come un oggetto variabile e cercare, tramite l'ottimizzazione, i parametri che la rendono più performante nell'MDP dato.

Per fare ciò, è necessario poter modificare la policy in modo continuo e calcolare come piccole variazioni dei suoi parametri influenzano il rendimento. Si introduce quindi il concetto di **policy parametrizzata**, che consente di usare metodi di ottimizzazione basati sul gradiente.

Definizione 6 (Policy parametrizzata). Sia $\Theta \subseteq \mathbb{R}^d$ uno spazio di parametri. Una **policy parametrizzata** è una funzione

$$\pi : S \times A \times \Theta \rightarrow [0, 1], \quad (s, a, \theta) \mapsto \pi(a|s; \theta),$$

differenziabile rispetto a θ , tale che per ogni $\theta \in \Theta$ e stato $s \in S$ la mappa $a \mapsto \pi(a|s; \theta)$ è una distribuzione di probabilità sulle azioni, ossia,

$$\sum_{a \in A} \pi(a|s; \theta) = 1.$$

In pratica, $\pi(a|s; \theta)$ è spesso implementata come una **rete neurale** che prende in ingresso lo stato e restituisce i parametri di una distribuzione sulle azioni.

Definizione 7 (Rendimento atteso). Dato un MDP M e una policy parametrizzata π_θ , il **rendimento atteso** è definito come

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [R_0] = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} \right].$$

Sul piano teorico, i metodi policy gradient consistono nell'aggiornare i parametri seguendo il gradiente esatto del rendimento atteso, cioè la direzione di massima crescita di $J(\theta)$. Questo fornisce il quadro concettuale: la policy è considerata una mappa differenziabile e l'ottimizzazione procede lungo il suo gradiente. Un esempio concreto è l'aggiornamento tramite **stochastic gradient descent (SGD)**:

$$\theta_{n+1} = \theta_n + \eta \nabla_\theta J(\theta_n).$$

Nella pratica, però, il calcolo esatto del gradiente richiede un'aspettativa su tutte le traiettorie dell'MDP, operazione impossibile da svolgere analiticamente o numericamente. Per superare questa difficoltà si sostituisce tale aspettativa con una stima ottenuta dai dati raccolti interagendo con l'ambiente (episodi). Si ottiene così uno *stimate del gradiente*, costruito a partire dai campioni disponibili, che viene utilizzato nell'aggiornamento:

$$\theta_{n+1} = \theta_n + \eta \nabla_\theta \hat{J}(\theta_n).$$

Questa sostituzione introduce varianza e rende necessarie strategie dedicate per controllarla e stabilizzare l'apprendimento, il **PPO** è stato progettato proprio con questo scopo.

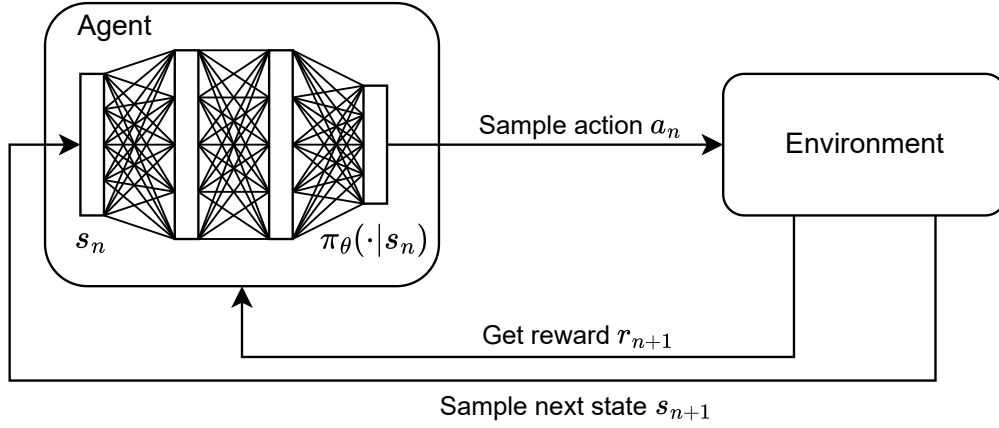


Figura 1: Schema ambiente-agente con multilayer perceptron come policy parametrizzata.

4 Proximal Policy Optimization

L'aggiornamento dei parametri con il gradiente stimato può produrre variazioni troppo ampie della policy, causando instabilità e degradando rapidamente le prestazioni. Per affrontare questo problema, PPO parte dal seguente teorema.

Teorema 1 (Gradiente del rendimento con vantaggio). Sia $M = (S, A, P, r, \gamma)$ un MDP, sia $\pi_\theta(a|s)$ una policy parametrizzata, e sia q una distribuzione iniziale sugli stati S . Allora il gradiente del rendimento atteso è dato da

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^{\infty} \nabla_\theta \log \pi_\theta(a_t|s_t) A^{\pi_\theta}(s_t, a_t) \right],$$

Dimostrazione: vedi Appendice A.1.

Durante l'addestramento, si interagisce con l'ambiente usando la policy corrente π_{θ_n} e si raccolgono traiettorie $(\dots, s_t, a_t, r_{t+1}, s_{t+1}, \dots)$. Una volta terminata la raccolta i dati osservati vengono usati per aggiornare i parametri verso una nuova π_θ .

Definizione 8 (Rapporto di probabilità). Il **rapporto di probabilità** misura come la nuova policy valuta un'azione rispetto alla policy vecchia:

$$\rho(s, a; \theta) = \frac{\pi_\theta(a|s)}{\pi_{\theta_n}(a|s)}.$$

Definizione 9 (Surrogato del policy gradient). Il **surrogato** usato per aggiornare i parametri con i dati raccolti sotto π_{θ_n} è

$$\mathcal{L}^{PG}(\theta) = \mathbb{E}_{\pi_{\theta_n}} \left[\sum_{t=0}^{\infty} \rho(s_t, a_t; \theta) A^{\pi_{\theta_n}}(s_t, a_t) \right].$$

Teorema 2 (Equivalenza del gradiente al punto θ_n). Vale

$$\nabla_{\theta} \mathcal{L}^{PG}(\theta_n) = \nabla_{\theta} J(\theta_n).$$

Dimostrazione: vedi Appendice A.2.

Poiché $A^{\pi_{\theta_n}}$ non è noto esattamente, lo si sostituisce con una stima $\hat{A}_t$ ottenuta dalle traiettorie raccolte. Massimizzare direttamente $\mathcal{L}^{PG}$ può spingere $\rho_t(\theta)$ lontano da 1, generando grandi cambiamenti della policy e instabilità. Per ovviare al problema, PPO utilizza una funzione obiettivo *clipped*.

Definizione 10 (Funzione Obiettivo PPO). In PPO si sostituisce ogni termine del surrogato con la versione *clipped*, limitando implicitamente la variazione della policy:

$$\mathcal{L}^{\text{PPO}}(\theta) = \mathbb{E}_{\pi_{\theta_n}} \left[\min \left(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right],$$

dove $\epsilon \in [0.1, 0.3]$ controlla l'ampiezza massima ammessa.

Il clipping preserva la direzione locale del gradiente nel punto corrente, ossia,

$$\nabla \mathcal{L}^{\text{PPO}}(\theta_n) = \nabla \mathcal{L}^{PG}(\theta_n)$$

ma riduce gli aggiornamenti eccessivi quando $|\rho_t(\theta) - 1|$ cresce. L'aggiornamento dei parametri è quindi

$$\theta_{n+1} = \arg \max_{\theta} L^{\text{CLIP}}(\theta),$$

realizzato con ottimizzatori a gradiente (spesso Adam) riutilizzando lo stesso insieme di dati per più epoche e suddividendolo in mini-batch.

A Dimostrazioni dei teoremi

A.1 Dimostrazione del Teorema Policy Gradient con vantaggio

Dimostrazione. Per definizione

$$J(\theta) = \sum_{\tau} \mathbb{P}_{\theta}(\tau) R_0(\tau), \quad \mathbb{P}_{\theta}(\tau) = \rho(s_0) \prod_{t \geq 0} \pi_{\theta}(a_t | s_t) P(s_{t+1} | s_t, a_t).$$

Poiché ρ e P non dipendono da θ ,

$$\nabla_{\theta} \log \mathbb{P}_{\theta}(\tau) = \sum_{t=0}^{\infty} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t).$$

Usando $\nabla f = f \nabla \log f$:

$$\nabla_{\theta} J(\theta) = \sum_{\tau} \mathbb{P}_{\theta}(\tau) \nabla_{\theta} \log \mathbb{P}_{\theta}(\tau) R_0(\tau) = \mathbb{E} \left[\sum_{t=0}^{\infty} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R_0 \right].$$

Separiamo il ritorno:

$$R_0 = C_t + \gamma^t R_t, \quad C_t = \sum_{k=0}^{t-1} \gamma^k r_{k+1}.$$

Il termine C_t non dipende da a_t , quindi

$$\mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(a_t|s_t) C_t \mid s_t] = 0,$$

poiché

$$\sum_a \pi_{\theta}(a|s_t) \nabla_{\theta} \log \pi_{\theta}(a|s_t) = \nabla_{\theta} \sum_a \pi_{\theta}(a|s_t) = \nabla_{\theta} 1 = 0.$$

Rimane

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_{t=0}^{\infty} \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \gamma^t R_t \right] = \mathbb{E} \left[\sum_{t=0}^{\infty} \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \gamma^t Q^{\pi_{\theta}}(s_t, a_t) \right].$$

Sottraendo una baseline $V^{\pi_{\theta}}(s_t)$ (che non cambia l'atteso),

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_{t=0}^{\infty} \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \gamma^t A^{\pi_{\theta}}(s_t, a_t) \right].$$

□

A.2 Dimostrazione dell'Equivalenza del gradiente al punto θ_n

Dimostrazione. Per definizione

$$\mathcal{L}^{PG}(\theta) = \mathbb{E}_{\pi_{\theta_n}} \left[\sum_{t=0}^{\infty} \rho(s_t, a_t; \theta) A^{\pi_{\theta_n}}(s_t, a_t) \right], \quad \rho(s_t, a_t; \theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_n}(a_t|s_t)}.$$

Derivando:

$$\nabla_{\theta} \rho(s_t, a_t; \theta) = \rho(s_t, a_t; \theta) \nabla_{\theta} \log \rho(s_t, a_t; \theta) = \rho(s_t, a_t; \theta) \nabla_{\theta} \log \pi_{\theta}(a_t|s_t).$$

Quindi

$$\nabla_{\theta} \mathcal{L}^{PG}(\theta) = \mathbb{E}_{\pi_{\theta_n}} \left[\sum_{t=0}^{\infty} \rho(s_t, a_t; \theta) \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) A^{\pi_{\theta_n}}(s_t, a_t) \right].$$

Valutando in $\theta = \theta_n$ si ha $\rho(s_t, a_t; \theta_n) = 1$, quindi

$$\nabla_{\theta} \mathcal{L}^{PG}(\theta_n) = \mathbb{E}_{\pi_{\theta_n}} \left[\sum_{t=0}^{\infty} \nabla_{\theta} \log \pi_{\theta_n}(a_t|s_t) A^{\pi_{\theta_n}}(s_t, a_t) \right] = \nabla_{\theta} J(\theta_n).$$

□